

Ce que tu dois savoir sur ce projet (Le RAG)

Ton projet utilise une architecture **RAG** (Retrieval-Augmented Generation). Au lieu de se fier uniquement à sa "mémoire" d'entraînement, l'IA utilise ton PDF comme une **source de vérité externe**.

1. **Les Embeddings (Le cœur)** : Le modèle all-MiniLM-L6-v2 transforme les phrases en listes de nombres (vecteurs). Si deux phrases ont un sens proche, leurs nombres seront mathématiquement proches.
2. **La Base Vectorielle (ChromaDB)** : C'est ta bibliothèque. Elle stocke ces nombres pour retrouver instantanément le bon passage du PDF quand tu poses une question.
3. **Le LLM (Llama 3)** : Il joue le rôle de "lecteur expert". Il reçoit ta question + les extraits du PDF trouvés par ChromaDB, puis il rédige une réponse fluide.

C'est une excellente approche. Pour ton rapport de stage, il est crucial de montrer que tu ne t'es pas contenté de "faire marcher le code", mais que tu maîtrises l'**architecture logicielle**.

Voici une comparaison détaillée et une fiche technique des outils.

1. Comparaison Stratégique : MonProjetIA vs. Projet RAG Expert

Dimension	MonProjetIA (Chatbot Simple)	Projet RAG Expert (Système RAG)
Architecture	Linéaire : Question → LLM → Réponse.	Boucle de recherche : Question → Recherche Documentaire → Augmentation du Prompt → LLM → Réponse.
Niveau de Spécialisation	Généraliste : Il connaît "un peu de tout" jusqu'à sa date d'entraînement.	Spécialiste : Il devient expert du document que tu lui donnes (ex: GIEC 2026).
Gestion des Hallucinations	Élevée : Si l'IA ne sait pas, elle invente une réponse plausible.	Faible : Le système force l'IA à dire "Je ne sais pas" si l'info n'est pas dans le PDF.
Limite de Mémoire	Limitée par la "fenêtre de contexte" du modèle (impossible de lui lire 500 pages d'un coup).	Illimitée : On peut indexer des milliers de pages, il ne récupère que les 3-4 paragraphes utiles.

2. Le "Stack" Technique (Les Outils)

C'est ici que tu montres ton expertise technique. Voici le rôle de chaque composant de ton projet actuel :

LangChain (L'Orchestrateur)

- **Rôle** : C'est le "cadre de travail" (framework). Il sert de colle entre tous les autres outils.
- **Pourquoi ?** Sans LangChain, tu devrais écrire des centaines de lignes de code pour gérer manuellement la lecture des PDF, le découpage du texte et l'envoi des données au modèle.

PyPDF (Le Loader)

- **Rôle** : C'est l'outil qui "casse" la structure visuelle d'un PDF pour en extraire le texte brut.
- **Pourquoi ?** Un PDF est un format complexe (images, colonnes, polices). PyPDF transforme cela en chaînes de caractères que Python peut traiter.

RecursiveCharacterTextSplitter (Le découpeur)

- **Rôle** : Il découpe le texte en morceaux (chunks) de 1000 caractères avec un chevauchement (overlap).
- **Pourquoi ?** Un LLM ne peut pas "digérer" un document entier d'un coup. Le découper permet de ne lui envoyer que les morceaux pertinents. Le chevauchement évite de couper une phrase importante en plein milieu.

HuggingFace Embeddings (Le Traducteur Mathématique)

- **Modèle utilisé** : all-MiniLM-L6-v2.
- **Rôle** : Il transforme le texte en **Vecteurs** (des listes de nombres).
- **Concept clé** : C'est ce qui permet la **recherche sémantique**. Au lieu de chercher des mots-clés exacts (comme Google), il cherche des "idées". Si tu demandes "pollution", il peut trouver des passages sur "émissions de CO2" car leurs vecteurs sont proches.

ChromaDB (La Base de Données Vectorielle)

- **Rôle** : C'est ton "index intelligent". Il stocke les vecteurs générés par HuggingFace.
- **Pourquoi ?** Contrairement à une base de données classique (SQL), ChromaDB est optimisée pour calculer très vite la distance entre deux vecteurs. C'est elle qui trouve "les 3 meilleurs morceaux du PDF" pour répondre à ta question.

Ollama & Llama 3 (Le Moteur de Génération)

- **Rôle** : Le LLM qui transforme l'information brute en une réponse humaine.
 - **Pourquoi ?** Llama 3 est l'un des modèles "Open Source" les plus puissants au monde. Ollama permet de le faire tourner **localement** sur ton PC sans utiliser de serveur externe (confidentialité totale).
-

3. Schéma de ton Workflow Actuel

1. **Ingestion** : PyPDF lit le document.
2. **Chunking** : TextSplitter découpe le texte.
3. **Embedding** : HuggingFace transforme les morceaux en nombres.
4. **Indexing** : ChromaDB enregistre tout ça dans le dossier db_expert.
5. **Retrieval** : Tu poses une question, ChromaDB sort les passages les plus proches.
6. **Augmentation** : On donne au LLM la question + les passages.
7. **Génération** : Llama 3 rédige la réponse finale.

Est-ce que ces explications t'aident pour ton rapport ? Si tu veux, je peux t'aider à rédiger une conclusion qui explique l'intérêt d'un tel système pour une entreprise.

C'est la **mémoire de recherche** de ton IA.

Son rôle se résume en trois points :

1. **Stockage intelligent** : Il contient ton PDF découpé en milliers de coordonnées mathématiques (les vecteurs).
2. **Gain de temps** : Il évite à l'IA de relire et d'analyser le PDF à chaque fois que tu lances le script.
3. **Recherche rapide** : Quand tu poses une question, le système fouille dans ce dossier pour extraire uniquement les 2 ou 3 passages les plus pertinents pour répondre.

En bref : C'est le "classeur" où ton programme range tout ce qu'il a appris de ton document.